

Object-Oriented Python

Day 2

Prep materials — Anthropic Fellows OA

August 2026

Contents

Day 2 — Object-Oriented Python (60 min)	1
1. Classes from zero (12 min)	1
2. Dunder methods worth knowing (8 min)	2
3. Properties (8 min)	2
4. Dataclasses — what you'll mostly see (12 min)	3
5. Inheritance, briefly (5 min)	3
6. Reading an unfamiliar OOP codebase fast (10 min) — DO THIS	4
7. Wrap-up (5 min)	4

Day 2 — Object-Oriented Python (60 min)

The test hands you a pre-built class hierarchy and asks you to extend it. So this session has two halves: writing classes, and **reading** someone else's fast. Type everything.

1. Classes from zero (12 min)

```
class Worker:
    """Docstring."""

    def __init__(self, index, capacity):  # constructor
        self.index = index                # instance attributes
        self.capacity = capacity
        self.running = []                # created fresh per instance

    def add(self, req):                   # `self` is ALWAYS explicit
        self.running.append(req)

    def load(self):
        return sum(r.size for r in self.running)

    def __repr__(self):                  # what print() shows
        return f"Worker({self.index}, load={self.load()})"

w = Worker(0, 100)
w.add(req)
print(w.load())                        # method — needs parens
```

Key differences from other languages:

- **No new** — call the class: `Worker(0, 100)`.
- **self is explicit** in every method definition, but not at the call site.
- **No private/public**. A leading underscore (`_cache`) means “internal, by convention.” Two underscores (`__x`) triggers name mangling — rarely used.
- **No overloading**. One `__init__` per class; use default arguments instead.
- Attributes are created by assignment — `self.anything = 1` just works, which means typos silently create new attributes rather than erroring.

GOTCHA — class attributes are shared across all instances:

```
class Bad:
    items = []                # ONE list for the whole class
    def __init__(self):
        self.count = 0       # per instance — correct

a, b = Bad(), Bad()
a.items.append(1)
b.items                    # [1] — surprise
```

Rule: anything mutable belongs in `__init__`, assigned to `self`.

2. Dunder methods worth knowing (8 min)

```
class Req:
    def __init__(self, id): self.id = id
    def __repr__(self): return f"Req({self.id})"    # debugging
    def __eq__(self, o): return self.id == o.id    # ==
    def __hash__(self): return hash(self.id)       # needed for set/dict
    def __lt__(self, o): return self.id < o.id     # enables sorting
    def __len__(self): return 1                   # len()
```

GOTCHA: defining `__eq__` without `__hash__` makes the class **unhashable** — it can no longer go in a set or be a dict key. And without `__lt__`, pushing objects into a heap crashes on ties. That’s why you push `(key, id, obj)` tuples instead of bare objects.

3. Properties (8 min)

A method that’s accessed like an attribute — very common in the codebases these tests give you.

```
class Request:
    def __init__(self, prompt_tokens, max_new_tokens):
        self.prompt_tokens = prompt_tokens
        self.max_new_tokens = max_new_tokens

    @property
    def reserved(self):                # computed on every access
        return self.prompt_tokens + self.max_new_tokens

r.reserved                # 12 — NO parentheses
r.reserved()              # TypeError: 'int' object is not callable
```

When you read an unfamiliar class, **check whether each member is a `@property` or a method** before you call it. Getting this wrong produces a confusing error at a distance and is a classic

10-minute time sink.

4. Dataclasses — what you'll mostly see (12 min)

Boilerplate-free classes for holding data. `__init__`, `__repr__`, and `__eq__` are generated for you.

```
from dataclasses import dataclass, field
from typing import Optional

@dataclass
class Request:
    id: int                    # required, positional
    arrival: int
    prompt_tokens: int
    tokens_generated: int = 0  # defaults AFTER non-defaults
    finished_at: Optional[int] = None
    log: list = field(default_factory=list) # mutable default -> factory!

    @property
    def done(self):
        return self.finished_at is not None

    def fresh(self):           # explicit reset copy
        return Request(self.id, self.arrival, self.prompt_tokens)

r = Request(1, 0, 8)
r = Request(id=1, arrival=0, prompt_tokens=8) # keywords also work
print(r)    # Request(id=1, arrival=0, ...) - free __repr__
```

Three rules that cause real bugs:

1. `log: list = []` raises `ValueError` in a dataclass — you must use `field(default_factory=list)`. (Plain classes let the same mistake pass silently, which is worse.)
2. Fields with defaults must come after fields without.
3. `@dataclass` sets `eq=True`, which sets `__hash__ = None` → **unhashable**. Use `@dataclass(frozen=True)` for a hashable, immutable version, or key collections by `.id`.

Mutability matters for this test: dataclass instances are mutable by default, so a function that mutates the requests it's given will corrupt a second run over the same objects. That's exactly why a `fresh()` helper exists.

5. Inheritance, briefly (5 min)

```
class Scheduler:
    def __init__(self, config): self.config = config
    def pick(self, queue): raise NotImplementedError # abstract-ish

class FifoScheduler(Scheduler):
    def __init__(self, config, quantum):
        super().__init__(config) # call the parent constructor
        self.quantum = quantum
    def pick(self, queue):       # override
        return queue[0]

isinstance(s, Scheduler) # True
```

You rarely need deep hierarchies in these tests, but you must recognize `super()`, overriding, and `NotImplementedError` stubs — the stubs are literally what you're asked to fill in.

6. Reading an unfamiliar OOP codebase fast (10 min) — DO THIS

This is the actual exam skill. Open `mock_oa/engine.py` and give yourself **8 minutes** to answer, in writing:

1. What classes exist, and what does each one *own* (which data)?
2. For each member: is it a field, a `@property`, or a method?
3. Which state is set for me, and which state am **I** responsible for updating? (In `engine.py` the docstring says this explicitly — most real codebases won't.)
4. What helper methods already exist that I'd otherwise rewrite by hand?
5. Where's the mutation risk — what happens if I run a function twice on the same objects?

Then check yourself: `Request.reserved` is a property (no parens); `Worker.has_room_for()` is a method that already implements the fit check; runtime fields like `finished_at` start as `None` and are yours to set; `fresh()` exists precisely because reuse would otherwise corrupt a rerun.

Habit to carry into the test: before writing any code, write a 5-line map of the given classes. Ten minutes spent here saves thirty later — the Reddit complaints were almost entirely about people skipping this step.

7. Wrap-up (5 min)

Do drills `c9` and `c10` in `day1_drills.py` (the `Task` dataclass and the `Pipeline` class). They're small on purpose — the goal is producing class scaffolding without pausing to recall syntax.

Tomorrow: the timed mock. Read `README.md`, then `engine.py`, then `tests.py` — 10 minutes before touching code.